DISTINCT INFORMATION-PROCESSING ROLES BY RELIABLE AND UNRELIABLE SPIKES IN BIOLOGICAL NEURAL NETWORKS

ZACH MOBILLE

1. Introduction

A system is considered reliable if it responds identically to the same input, even when initialized in different states. The degree to which a system is reliable influences its ability to process information. If robust representation of the external input as an internal state is of priority, than a reliable response whereby this input always converges to the same internal state is beneficial. However, if the information-processing task is better characterized as distinguishing between subtle differences in past input, then unreliable responses where each initial state results in a distinct response are preferred. The nervous system must perform both information-processing tasks, and is subject to a large variety of noise [1] that produces unreliability. Whether this "noise" in the nervous system contains valuable information is a subject of ongoing debate. Here, I focus exclusively on the effect of noise in the initial conditions of spiking neural networks on reliability.

The reliability of a system is intimately linked to its dynamical stability. Previous work has shown that the reliability of biological neural networks depends on the dynamics of their constituents, the connectivity between them, and the external input given to the network [2]. These factors shape the dynamical stability of the network, as characterized by its Lyapunov spectrum, and thus determine its reliability. Chaotic networks of spiking neurons can display regimes of coexisting reliability and unreliability in the same response to a given stimulus [3]. Although clearly implicated in information processing, the concrete role of reliable versus unreliable spikes has never been established in a quantitative framework. Here, I use information theoretic measures to identify the distinct roles that reliable spikes and unreliable spikes play in information processing of inputs to biological neural networks. Motivated by the discussion in Ref. [3] and the differences in dynamical mechanism producing them, I hypothesize that reliable spikes primarily store information about the current input to a network while unreliable spikes are more informative about past inputs.

2. Mathematical Models and Methods

2.1. **Balanced network model.** The ith neuron in the network model of $N = 1000$ neurons I focused on for the numerical reproduction is described by the following stochastic differential equation [3]:

$$\mathrm{d}\theta_i = \left[F(\theta_i) + Z(\theta_i) \left(\eta + \sum_{j=1}^{N} a_{ji} g(\theta_j) \right) + \frac{\epsilon^2}{2} Z(\theta_i) Z'(\theta_i) \right] \mathrm{d}t + \epsilon Z(\theta_i) \mathrm{d}W_{i,t} \tag{1}$$

where $\theta_i \in [0, 1)$ is the phase of the ith neuron ($i = 1, \ldots, N$), η is a constant input, a_{ji} is the synaptic weight of the connection from neuron j to neuron i, ϵ scales the noisy time-dependent input to the system given by the independent Wiener process $W_{i,t}$. Furthermore,

the functions

$$F(\theta_i) = 1 + \cos(2\pi\theta_i)$$
$$Z(\theta_i) = 1 - \cos(2\pi\theta_i)$$

represent the intrinsic dynamics and stimulus response curve, respectively. They are shown plotted in figure 1.

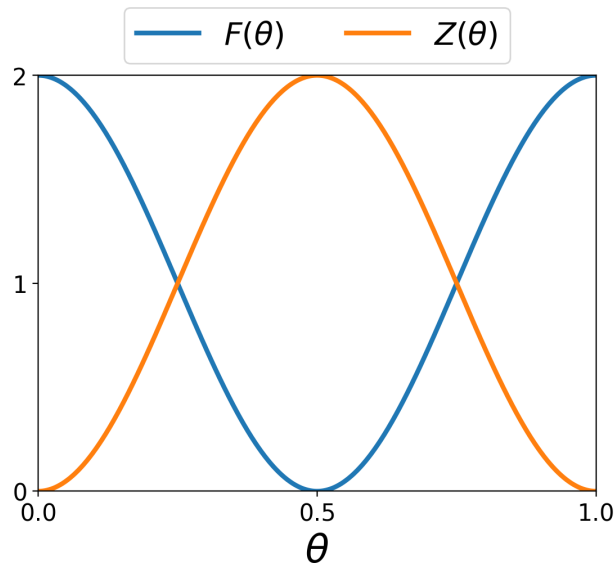

FIGURE 1. Depiction of the intrinsic dynamics $F(\theta)$ and stimulus response curve $Z(\theta)$ of the theta-neuron model studied here. These functions come from the fact that this model is equivalent to the quadratic integrate-and-fire (QIF) model of voltage v after a change of variable and taking the limit of infinite reset and peak voltage $v_r, v_p \to \pm\infty$. The function $F(\theta)$ makes it so that the neuron is "type 1," meaning the transition from no spiking to spiking is described by a saddle-node bifurcation. The function $Z(\theta)$ renders the response of the system to inputs dependent on its current state.

The function

$$g(\theta) = \begin{cases} d\left\{b^2 - \left[(\theta + \frac{1}{2}) \bmod 1 - \frac{1}{2}\right]^2\right\}^3, & \theta \in [-b, b] \\ 0, & \text{otherwise} \end{cases}$$

is bump-shaped and mediates coupling between neurons by multiplying the synaptic weights; its parameters are taken as $d = 35/32$ and $b = 1/20$. A plot of the coupling function $g(\theta)$ is shown in figure 2.

A neuron is said to "spike" when its phase θ exceeds 1, at which point it is reset to 0. Thus, when a spike occurs, $g(\theta)$ is maximized and that neuron affects all of its neighbors through the weighted adjacency matrix $A = \{a_{ij}\}$. For the "balanced network" regime studied in the numerical reproduction here, 80% of the neurons are excitatory and 20% of the neurons are inhibitory. Quantitatively, this means that 80% of the rows of the adjacency matrix A are completely positive (excitatory) and the other 20% are completely negative (inhibitory). On average, each neuron has $K = 20$ incoming connections from each population (excitatory

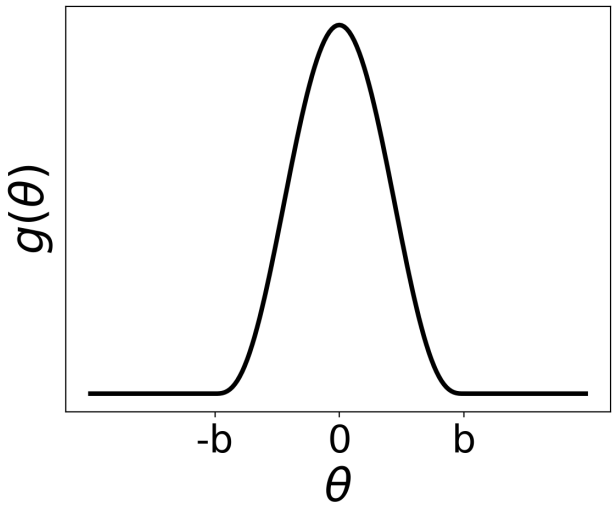

FIGURE 2. Coupling function $g(\theta)$. Through use of this function, pulse-like synaptic events are modeled in the network of theta neurons described by equation 1.

or inhibitory) and its synaptic weights scale like $O(1/\sqrt{K})$. A realization of the random network connectivity, as encapsulated by the weighted adjacency matrix A, is visualized in figure 3.

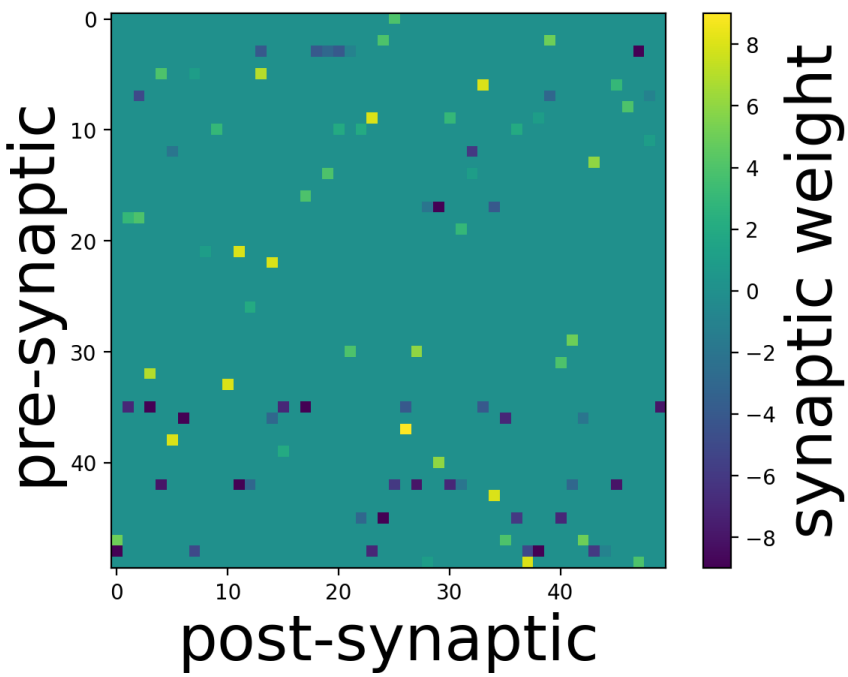

FIGURE 3. Illustration of the weighted adjacency matrix from the model described by equation 1, for $N = 50$ neurons and $K = 1$ incoming connection from each population. This connection matrix was created using the function `make_A` in the python code of appendix section 6.8.

2.2. **Two-neuron recurrent network.** As a minimal model of spike reliability, in section 4 I study a special case of equation 1 with $N = 2$ neurons and simplified intrinsic dynamics:

$$\dot{\theta}_1 = \omega_1 + a_{\text{fb}} Z(\theta_1) g(\theta_2) + \epsilon Z(\theta_1) I(t) \tag{2a}$$

$$\dot{\theta}_2 = \omega_2 + a_{\text{ff}} Z(\theta_2) g(\theta_1) \tag{2b}$$

the parameters ω_1 and ω_2 represent the natural frequencies of neurons 1 and 2. In the absence of external inputs, these are the frequencies at which they would oscillate. The parameter a_{fb} represents the strength of the feedback connection from neuron 2 to neuron 1, whereas a_{ff} represents the strength of the feedforward connection from neuron 1 to neuron 2. The "forward" and "back" are with respect to the external stimulus $I(t)$, which is a white noise process scaled by ϵ and fed exclusively to θ_1. The functions $Z(.)$ and $g(.)$ are as described in section 2.1.

A numerical solution to equation 2 is shown below in figure 4. Each time neuron 2 spikes, a contribution $a_{\text{fb}} Z(\theta_1) g(0)$ is added to neuron 1's dynamics. Similarly, each time that neuron 1 spikes, the contribution $a_{\text{ff}} Z(\theta_2) g(0)$ is added to neuron 2's phase velocity. Since $Z(\theta) = 1 - \cos(2\pi\theta)$ is maximized at $\theta = 0.5$, these contributions are most significant when the post-synaptic neuron is halfway through its phase. If the post-synaptic neuron is about to spike (its θ is slightly less than one) or has just spiked (its θ is slightly greater than zero), an incoming spike will have little effect due to the fact that $Z(\theta) \to 0$ as $\theta \to 0, 1$ (see orange curve in figure 1). The paradigm described above implies that the neurons only interact when they spike. This is well-suited to the biological scenario where the neurons are separated from another by a distance exceeding ≈1 mm and thus do not communicate significant information via subthreshold voltage oscillations [4]. The precise value of the distance over which subthreshold oscillations attenuate significantly can be inferred from the length constant of the neuron, which depends on its geometry and electrical properties [5].

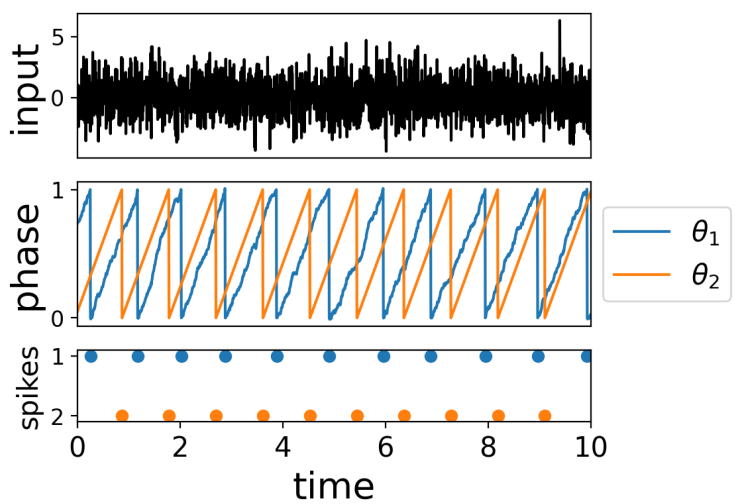

FIGURE 4. Simulation of equation 2, solved numerically using the forward Euler-Maruyama method with time step size $\Delta t = 0.005$. The dots in the bottom raster plot denote the times when each neuron spiked. This figure was generated by the code in appendix section 6.1.

In Ref. [2], the authors introduce the dynamical system given by equation 2 as a minimal model of spike reliability. They relate this reliability to the system's dynamical stability with respect to parameters a_{ff}, a_{fb}, and ϵ by computing the maximum Lyapunov exponent for various combinations of these parameters. The specific choices of parameter values used here for the 2-oscillator system are guided by their results. In order to modulate the reliability of the spike train response, the noise level ϵ is tuned. Low values of ϵ result in consistently unreliable spike trains where the time of spikes is highly variable across trials in a constant fashion (see figure 5(A)). High values of the noise level ϵ result in highly reliable spike trains where the spike times are virtually identical on each trial (see figure 5(C). Intermediate values of ϵ result in the most interesting scenario where the reliability of spiking events changes over time, as visualized in figure 5(B). The intermediate regime depicted here is proximal in $(\epsilon, a_{\text{fb}})$ phase space to the "triple point" shown in figure 13 of Ref. [2], so-called due to the fact that the maximum Lyapunov exponent is positive, negative, and zero in a small neighborhood of this point. Functionally, this intermediate regime results in a response that is not totally unreliable nor totally reliable. Such a phenomenon was shown to occur by Ref. [3] in the more complicated balanced network model described here by equation 1. In their model, intermittent periods of reliable spiking interrupt unreliable spiking at the level of a single neuron due to the fact that reliability can propagate through nodes in the network when in a chaotic state. They show that this ultimately results from different neurons contributing to state space expansion by different amounts at different times. In the recurrent two-neuron model for which novel results are shown in section 4, this regime of unreliable-reliable spike coexistence is acquired by tuning ϵ so that the system is close to a transition between chaotic and not (see figure 5).

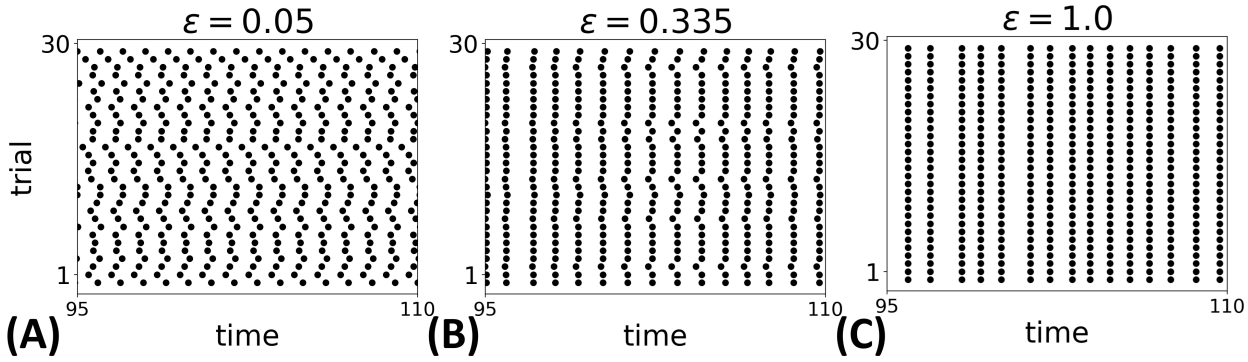

FIGURE 5. Demonstration of reliability versus unreliability. Shown are the spike time responses of neuron 1 of the model given by equation 2 to the same frozen white noise stimulus but initialized in different states. Each initial state represents a different trial. The noise level ϵ modulates the degree to which the response is reliable. (A) low reliability case, (B) intermediate reliability case, (C) high reliability case. The data from these plots were generated with the python function `two` shown in appendix section 6.2.

2.3. **Participation fraction and spike train partitioning.** In order to test the hypothesis that reliable and unreliable spikes play distinct information-processing roles, a method is needed for distinguishing between the two. The concept of the spike participation fraction

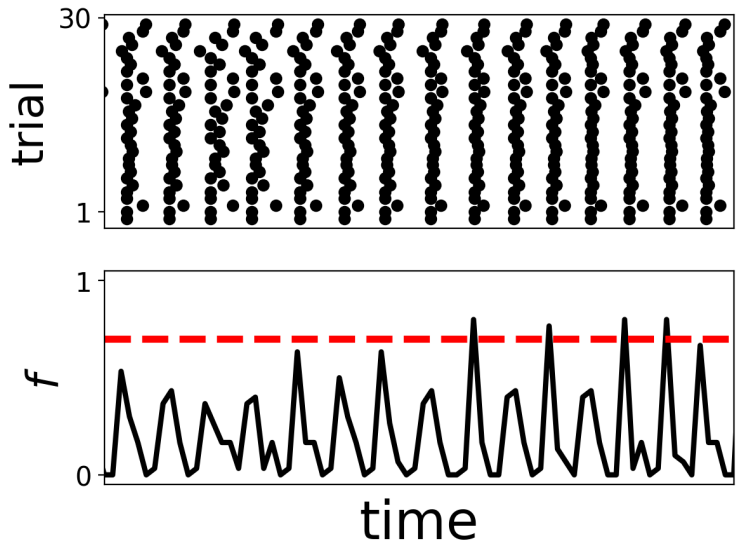

FIGURE 6. An example ensemble simulation of 30 trials (top) with the corresponding participation fraction f (bottom) obtained by counting trials in time bins of width $\Delta t = 0.2$, as described in the text. The horizontal dashed line represents the threshold of $f = 0.6$ used to distinguish reliable spikes from unreliable ones. This figure was generated by the code in appendix section 6.3.

f from Ref. [3] is used. The fraction f is obtained from spiking data over multiple trials by binning time, counting up the number of trials that have a spike within that time bin, and dividing by the total number of trials. By thresholding this quantity f, the spikes in a given trial can be distinguished as reliable or unreliable by evaluating whether f was above or below threshold at that time, respectively. This process is depicted in figures 6 and 7.

An illustration of the spikes in a given trial for a network in the regime of coexisting reliability and unreliability (as in figure 5(B)), after identifying them as either reliable or unreliable using the method described above, is shown in figure 7. From this result, it is clear that 1) unreliable spikes outnumber reliable ones and 2) reliable and unreliable spikes alternate in the same trial with overlap in time. This phenomenon of intermittent reliability amongst a relatively larger duration of unreliability is the same as that observed in Ref. [3] for the balanced network of $N = 1000$ spiking neurons. In order to define and leverage this intermittency for addressing my hypothesis, I construct the inter-spike interval (ISI) distribution for both reliable and unreliable spikes. I then partition the spike trains at spikes where the ISI is greater than a high percentile of its corresponding ISI distribution. The average firing rate of each partitioned spike train is then computed by counting the number of spike and dividing the duration of the spike train. Additionally, the average value of the white noise stimulus $\epsilon I(t)$ over this same duration is computed. These variables are considered as the response r and stimulus s in mutual information calculations shown in the novel results section 4.

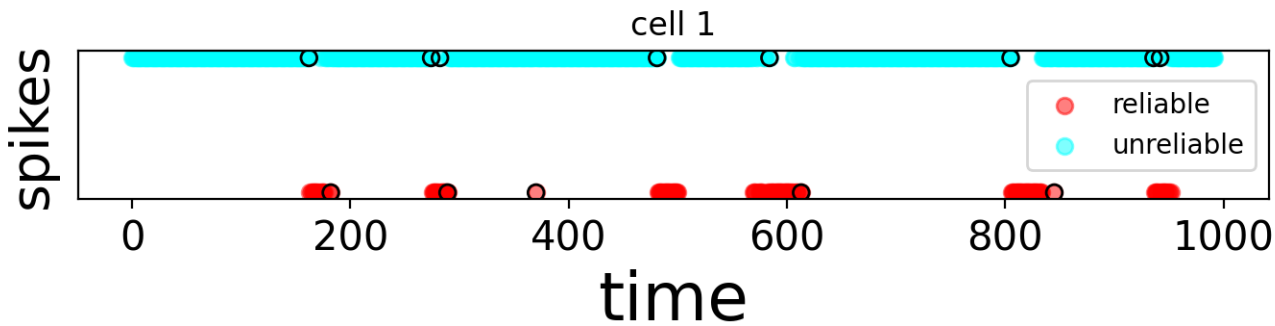

FIGURE 7. Identification of unreliable versus reliable spikes in a single trial from the participation fraction thresholded at $f = 0.6$ for a simulation with $\epsilon = 0.15$. The spikes circled in black were deemed points at which to partition one spike train from the next. This was done by selecting spikes for which the subsequent interspike interval (ISI) was greater than some high percentile of the ISI histogram for the entire reliable or unreliable time series. This figure was generated by the code in appendix section 6.4.

2.4. **Mutual information.** To quantify the information shared between spike train responses r and stimuli s, the mutual information between them is computed. Mutual information between two continuous random variables s and r is defined as

$$I_m(s,r) = \iint p(s,r)\log_2\frac{p(s,r)}{p(s)p(r)}\mathrm{d}r\mathrm{d}s$$

and measures the information gain about s resulting from obtaining information about r, or equivalently, vice versa. In the results shown in section 4, mutual information between the average white noise stimulus and the average firing rate of the reliable/unreliable spikes is estimated directly using the `mutual_info_regression` function from the Python package scikit-learn.

3. Numerical Reproduction

To obtain a baseline framework for reliability in a balanced network of spiking neurons, I simulated the system given by equation 1 and described above in section 2.1. Figure 1 of Ref. [3] was targeted, so as to recreate the essential network statistics and dynamics before addressing the main question of information processing by reliable and unreliable spikes. A successful reproduction of the balanced network of $N = 1000$ neurons is shown in figure 8 below.

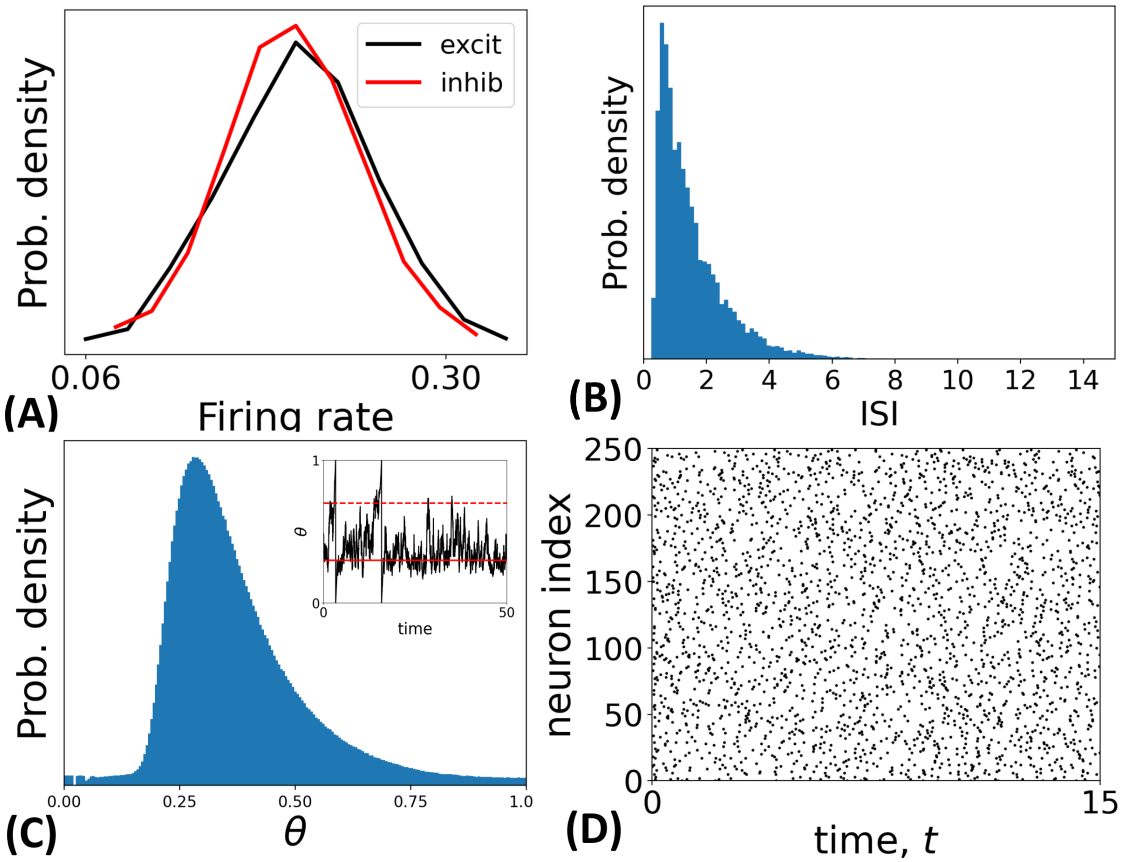

FIGURE 8. Reproduction of figure 1 from Ref. [3]. (A) Firing rate distribution of excitatory and inhibitory populations, (B) Inter-spike interval (ISI) distribution of the network simulation with an average coefficient of variation $\langle \mathrm{CV} \rangle \approx 1.3$, (C) Distribution of the state over neurons and time for a simulation of duration 50, (inset) an example time series of one of the neurons with stable (solid) and unstable (dashed) fixed points denoted by red lines, (D) an example raster plot of 250 of the 1000 neurons in the network simulation. These figures were produced by codes shown in appendix sections 6.9, 6.10, 6.11, 6.12, and 6.13.

Although the network statistics were almost perfectly reproduced (figure 8), I failed to obtain reliable and unreliable spike train responses by modulating ϵ in the full network model, as was done in Ref. [3]. One hint at why this failure might be occurring is in the discrepancy between my average coefficient of variation of $\langle \mathrm{CV} \rangle \approx 1.3$ and that found in Ref. [3] of $\langle \mathrm{CV} \rangle \approx 0.9$. Thus, at the demand of urgency and simplicity, I opted for the two-oscillator network stated in section 6.2 for my main results in the next section 4.

4. Novel Results

The primary hypothesis of the present work is that reliable and unreliable spikes play distinct roles in the information processing of incoming inputs. Specifically, it is predicted that reliable spikes mostly contain information about the concurrent stimulus whereas unreliable spikes are informative about past inputs or states of the network. This distinction

is motivated by the fact that unreliability is a consequence of chaos and sensitivity to initial conditions, whereas reliable spikes result from repeatable and consistent mappings from a given input to a single output.

The hypothesis stated above is addressed through the results shown in figures 9 and 10 below, where scatter plots of the average stimulus versus the spike train rate of reliable and unreliable responses are shown along with the mutual information estimated between these variables. In figure 9, the average value of the stimulus taken concurrently with the spike rate is shown. In this case, the mutual information between current stimulus and reliable spike rates (figure 9(A)) is higher than that between the current stimulus and unreliable spike rates (figure 9(B)). This result confirms that reliable spikes are more informative then unreliable spikes about current stimuli inpinged on the network. Figure 10 shows the same as figure 9, but for stimuli 9 spike trains previous to the spike train over which the rate is plotted on the horizontal axis. In this sense, we are looking at the relationship between past inputs and the current responses in figure 10. In particular, the mutual info between past inputs and reliable responses (figure 10(A)) is lower than that between past inputs and unreliable responses (figure 10(B)). Furthermore, there is an approximately ten-fold increase in the mutual information unreliable spikes carry about past inputs over the mutual information they carry about current inputs (MI=0.051 from figure 10(B) and MI=0.006 from figure 10(B)). Overall, these results verify the hypothesis that reliable spikes and unreliable spikes play distinct information-processing roles in neural networks.

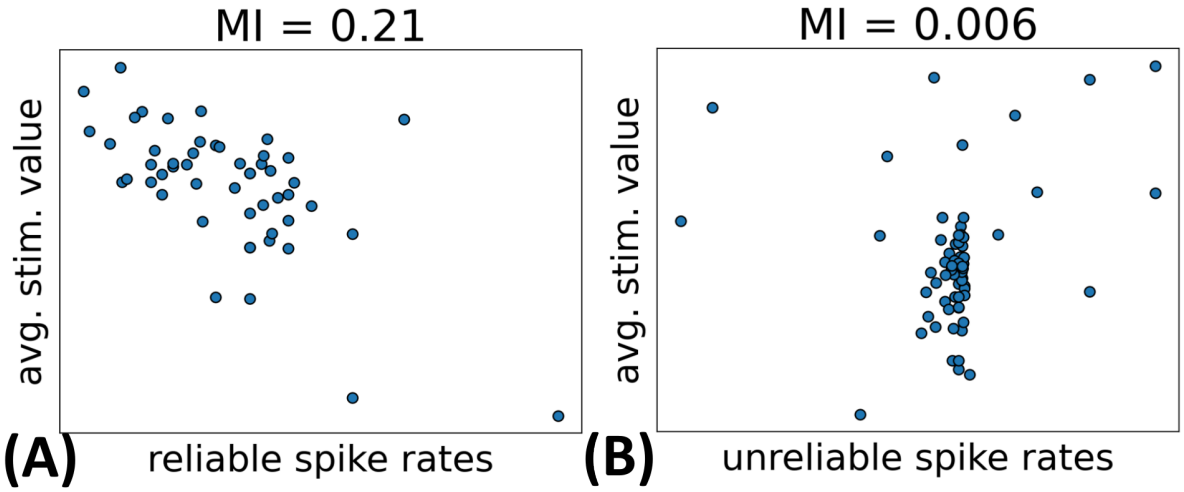

FIGURE 9. Mutual information results for spike rates versus the current average stimulus value. "MI" in the plot title stands for mutual information. The data shown in this plot are generated by first simulating 10000 time units of the two-oscillator model using the code in appendix section 6.2, distinguishing reliable and unreliable spikes using code from appendix sections 6.3 and 6.4, getting the respective rates and stimuli with code from appendix section 6.5. Finally, these data are put on a scatter plot and the mutual information is computed by the code in appendix section 6.6.

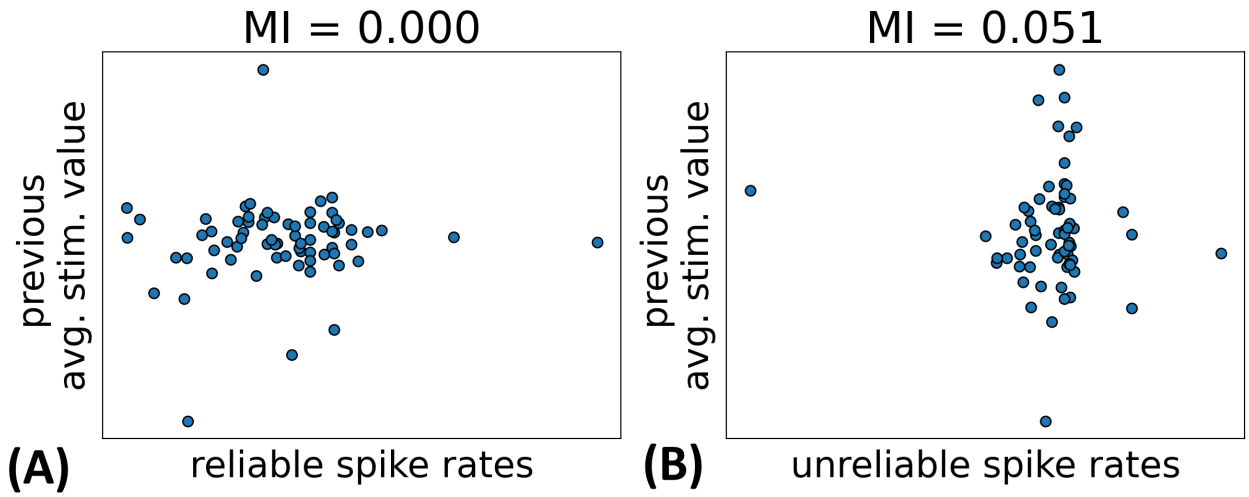

FIGURE 10. Mutual information results for spike rates versus the average stimulus value 9 spike trains in the past from the current spike train that the rate is computed with respect to. The data here are obtained in the same way as described in the caption of figure 9, but the mutual information and scatter plots are obtained using 6.7, instead.

5. Conclusion

Past work has shown that random networks of spiking neurons increase their reliability when connection probability is distance-dependent [6], rate models of neuronal networks in a coexisting unstable/stable regime optimize their capacity for sequential memories [7], and that network chaos is compatible with millisecond spike time precision [8]. However, none of this research has investigated the distinct role of unreliable versus reliable spikes in neuronal computation. This work seeks to fill that gap by considering a simple model of reliability in a two-neuron model and computing its mutual information with present and time-lagged inputs. The results obtained here suggest that optimal decoding schemes may be created by separately considering reliable versus unreliable spikes for predicting stimuli at different times in the past. In summary, the present work enriches the current understanding of how neural networks process information with their spikes in diverse manners.

6. Appendix

6.1. Python code: 2 oscillator trajectory to create figure 4.

```
import matplotlib.pyplot as plt
import numpy as np

def g(theta):
    b=1./20.
    d=35./32.
    if isinstance(theta, float):
        condition = (theta>-b) and (theta<b)
        if condition:
            return d*(b**2 - ((theta+.5) % 1 - .5)**2)**3
        else:
            return 0.
    else:
```

```
        ret = np.zeros(len(theta))
        for i in range(len(theta)):
            condition = (theta[i]>-b) and (theta[i]<b)
            if condition:
                ret[i] = d*(b**2 - ((theta[i]+.5) % 1 - .5)**2)**3
            else:
                ret[i] = 0.
        return ret

def z(theta):
    return (1-np.cos(2*np.pi*theta))/(2*np.pi)

# parameters
aff= 1.
afb = 1.5
w1 = 1
w2 = 1.1
epsilon=1.5

# Integration settings
dt=0.005          # Integration step size
tmax=10         # Total integration time
N=int(tmax/dt)   # Total number of steps

# Initialize vectors for storage of time-series
t = np.arange(0, tmax, dt)
phi1 = np.zeros(N) # phase 1
phi2 = np.zeros(N) # phase 2

# random initial conditions
np.random.seed()
phi1[0] = np.random.rand()
phi2[0] = np.random.rand()

np.random.seed(4)
I = np.random.normal(0, 1, N) # white noise input

spikeTimes1=[] # to store the time of each spike
spikeTimes2=[] # to store the time of each spike

# Step equation forward in time (Euler's method)
for n in range(N-1): # loop through time
    if phi1[n] >= 1.:
        phi1[n+1] = 0.
        spikeTimes1.append((n+1)*dt)
    else:
        dphi1dt = w1 + afb*z(phi1[n])*g(phi2[n]) + epsilon*I[n]
        phi1[n+1] = phi1[n] + dphi1dt*dt
    if phi2[n] >= 1:
        phi2[n+1] = 0.
        spikeTimes2.append((n+1)*dt)
    else:
        dphi2dt = w2 + aff*z(phi2[n])*g(phi1[n])
        phi2[n+1] = phi2[n] + dphi2dt*dt

fig, axes = plt.subplots(3,1,figsize = (5,4),gridspec_kw = {'height_ratios':[1,1,0.5]})
axes[0].plot(t, epsilon*I,color='black')
axes[0].set_ylabel('input',fontsize=20)
axes[0].set_xticks([])
axes[0].tick_params(axis='y',labelsize=12)
axes[0].set_xlim([0,tmax])
axes[1].set_ylabel('phase',fontsize=20)
axes[1].set_yticks([0,1])
```

```
axes[1].plot(t, phi1, label=r'$\theta_1$')
axes[1].plot(t, phi2, label=r'$\theta_2$')
axes[1].set_xticks([])
axes[1].tick_params(axis='y',labelsize=12)
axes[1].set_xlim([0,tmax])
axes[1].legend(loc='center right',bbox_to_anchor=(1.3, 0.5),fontsize=15)
axes[2].set_ylabel('spikes',fontsize=15)
axes[2].scatter(spikeTimes1,np.ones(len(spikeTimes1)))
axes[2].scatter(spikeTimes2,np.zeros(len(spikeTimes2)))
axes[2].set_yticks([0,1],[2,1])
axes[2].set_xlim([0,tmax])
axes[2].tick_params(axis='y',labelsize=12)
axes[2].set_ylim([-0.1,1.1])
axes[2].tick_params(axis='x',labelsize=15)
axes[2].set_xlabel('time',fontsize=20)
plt.savefig('traj.png',bbox_inches='tight',dpi=200)
plt.show()
```

6.2. Python code: 2 oscillator function for returning spike times.

```
import numpy as np
import matplotlib.pyplot as plt
from numba import jit

@jit
def two(I,eps,tf):
    def g(theta):
        b=1./20.
        d=35./32.
        if isinstance(theta, float):
            condition = (theta>-b) and (theta<b)
            if condition:
                return d*(b**2 - ((theta+.5) % 1 - .5)**2)**3
            else:
                return 0.
        else:
            ret = np.zeros(len(theta))
            for i in range(len(theta)):
                condition = (theta[i]>-b) and (theta[i]<b)
                if condition:
                    ret[i] = d*(b**2 - ((theta[i]+.5) % 1 - .5)**2)**3
                else:
                    ret[i] = 0.
            return ret

    def z(theta):
        return (1-np.cos(2*np.pi*theta))/(2*np.pi)

    # parameters
    aff= 1.
    afb = 1.5
    w1 = 1.
    w2 = 1.1
    epsilon=eps

    # Integration settings
    dt=0.005          # Integration step size
    tmax=tf         # Total integration time
    N=int(tmax/dt)   # Total number of steps

    # Initialize vectors for storage of time-series
    t = np.arange(0, tmax, dt)
    phi1 = np.zeros(N) # phase 1
```

```
    phi2 = np.zeros(N) # phase 2

    # random initial conditions
    #np.random.seed()
    phi1[0] = np.random.rand()
    phi2[0] = np.random.rand()
    #print(phi1[0])
    #print(phi2[0])

    #np.random.seed(4)
    #I = np.random.normal(0, 1, N) # white noise input
    #plt.plot(I)

    spikeTimes1=[] # to store the time of each spike
    spikeTimes2=[] # to store the time of each spike

    # Step equation forward in time (Euler's method)
    for n in range(N-1): # loop through time
        if phi1[n] >= 1.:
            phi1[n+1] = 0.
            spikeTimes1.append(n*dt)
            #spikeTimes1 = np.append(spikeTimes1,n*dt)
        else:
            dphi1dt = w1 + afb*z(phi1[n])*g(phi2[n]) #+ z(phi1[n])*epsilon*I[n]
            phi1[n+1] = phi1[n] + dphi1dt*dt + z(phi1[n])*epsilon*I[n]
        if phi2[n] >= 1:
            phi2[n+1] = 0.
            spikeTimes2.append(n*dt)
            #spikeTimes2 = np.append(spikeTimes2,n*dt)
        else:
            dphi2dt = w2 + aff*z(phi2[n])*g(phi1[n])
            phi2[n+1] = phi2[n] + dphi2dt*dt
    return spikeTimes1, spikeTimes2
```

6.3. Python code: makes figure 6.

```
import numpy as np
import matplotlib.pyplot as plt
from two import two

# Integration settings
dt=0.005          # Integration step size
tmax=1000         # Total integration time
N=int(tmax/dt)   # Total number of steps
I = np.random.normal(loc=0., scale=np.sqrt(dt), size=N) # white noise input
fig, axs = plt.subplots(2)
rng = np.array([165,180])
numTrials = 30
st = {}
for t in range(numTrials):
    eps=.335 # .
    spikeTimes1, spikeTimes2 = two(I,eps,tmax)
    spikeTimes1 = np.array(spikeTimes1)
    spikeTimes2 = np.array(spikeTimes2)
    st['t'+str(t)]=spikeTimes1
    axs[0].scatter(st['t'+str(t)],t*np.ones(len(st['t'+str(t)])),color='black')
axs[0].set_xlim(rng)
axs[0].set_ylabel('trial',fontsize=25)
axs[0].set_yticks([1,30])
axs[0].tick_params(axis='y',labelsize=15)
axs[0].set_xticks([])

binWidth=0.2
```

```
numBins=int(tmax/binWidth)
fVec=np.zeros(numBins)
time = 0
for b in range(numBins): # loop through time
    for t in range(numTrials): # loop through trials
        s = st['t'+str(t)][st['t'+str(t)]>time]
        s = s[s<time+binWidth]
        fVec[b] = fVec[b] + len(s) # add the number of spikes falling in the time window
    #print(time)
    time = time + binWidth
fVec = fVec/numTrials

axs[1].plot(fVec,linewidth=3,color='black')
rngadj=rng/binWidth
axs[1].set_xlim(rngadj)
axs[1].hlines(0.7,rngadj[0],rngadj[1],linestyle='--',color='red',linewidth=4)
axs[1].set_ylabel(r'$f$',fontsize=25)
axs[1].set_xticks([])
axs[1].set_yticks([0,1])
axs[1].set_xlabel('time', fontsize=30)
axs[1].tick_params(axis='y',labelsize=15)
plt.savefig('f.png',bbox_inches='tight',dpi=200)
plt.show()
```

6.4. Python code: makes figure 7.

```
import numpy as np
import matplotlib.pyplot as plt
from two import two

############## PRODUCE DATA ###############
# Integration settings
dt=0.005          # Integration step size
tmax=1000         # Total integration time
N=int(tmax/dt)    # Total number of steps
I = np.random.normal(loc=0., scale=np.sqrt(dt), size=N) # white noise input

numTrials = 30
st1 = {}
st2 = {}
for t in range(numTrials):
    eps=.15 # .33
    spikeTimes1, spikeTimes2 = two(I,eps,tmax)
    spikeTimes1 = np.array(spikeTimes1)
    spikeTimes2 = np.array(spikeTimes2)
    st1['t'+str(t)]=spikeTimes1
    st2['t'+str(t)]=spikeTimes2
binWidth=0.2
numBins=int(tmax/binWidth)
fVec1=np.zeros(numBins)
fVec2=np.zeros(numBins)
time = 0
for b in range(numBins): # loop through time
    for t in range(numTrials): # loop through trials
        s1 = st1['t'+str(t)][st1['t'+str(t)]>time]
        s1 = s1[s1<time+binWidth]
        s2 = st2['t'+str(t)][st2['t'+str(t)]>time]
        s2 = s2[s2<time+binWidth]
        fVec1[b] = fVec1[b] + len(s1) # add the number of spikes falling in the time window
        fVec2[b] = fVec2[b] + len(s2) # add the number of spikes falling in the time window
    #print(time)
    time = time + binWidth
fVec1 = fVec1/numTrials
```

```
fVec2 = fVec2/numTrials
rng = np.array([300,320])

fig, axs = plt.subplots(2)
for t in range(numTrials):
    axs[0].scatter(st1['t'+str(t)],t*np.ones(len(st1['t'+str(t)])),color='black')
axs[0].set_title(r'$\epsilon=$'+str(eps),fontsize=35)
axs[0].set_xlim(rng)
axs[0].set_ylabel('trial',fontsize=25)
axs[0].set_yticks([1,30])
axs[0].tick_params(axis='y',labelsize=15)
axs[0].set_xticks([])

axs[1].plot(fVec1,linewidth=3,color='black')
rngadj=rng/binWidth
axs[1].set_xlim(rngadj)
axs[1].hlines(0.7,rngadj[0],rngadj[1],linestyle='--',color='red',linewidth=4)
axs[1].set_ylabel(r'$f$',fontsize=25)
axs[1].set_xticks([])
axs[1].set_yticks([0,1])
axs[1].tick_params(axis='y',labelsize=15)
#plt.savefig('f1.png',bbox_inches='tight',dpi=200)
plt.show()
plt.close()

############################## distinguish unrel from rel #############
cutoff=0.6
stInd = np.random.randint(0,numTrials)
#len(st['t'+str(stInd)])
relSpks1 = [] # for storing the reliable spikes of cell 1
urelSpks1= [] # for storing the unreliable spikes of cell 1
# convert from bin of f to time
for sTime in range(len(st1['t'+str(stInd)])): # loop through all spike times in this train
    if int(sTime/binWidth) < len(fVec1):
        instFrac = fVec1[int(sTime/binWidth)] # instantaneous participation fraction at the time of this spike
        if instFrac > cutoff:
            relSpks1 = np.append(relSpks1,sTime)
        else:
            urelSpks1 = np.append(urelSpks1,sTime)
relSpks2 = [] # for storing the reliable spikes of cell 2
urelSpks2= [] # for storing the unreliable spikes of cell 2
for sTime in range(len(st2['t'+str(stInd)])): # loop through all spike times in this train
    if int(sTime/binWidth) < len(fVec2):
        instFrac = fVec2[int(sTime/binWidth)] # instantaneous participation fraction at the time of this spike
        if instFrac > cutoff:
            relSpks2 = np.append(relSpks2,sTime)
        else:
            urelSpks2 = np.append(urelSpks2,sTime)
# partition the spike trains
pRel = np.percentile(np.diff(relSpks1), 96) # return 96th percentile of ISI histogram for reliable spikes
j=0
tmpRel = []
for i in range(len(relSpks1)-1):
    if np.diff(relSpks1)[i] > pRel:
        tmpRel.append(relSpks1[i])

pUrel= np.percentile(np.diff(urelSpks1), 99) # return 99th percentile of unreliable ISI histogram
j=0
tmpUrel= []
for i in range(len(urelSpks1)-1):
    if np.diff(urelSpks1)[i] > pUrel:
        tmpUrel.append(urelSpks1[i])

```

```
plt.figure(figsize=(8, 1))
plt.scatter(relSpks1,np.zeros(len(relSpks1)),color='red',alpha=0.5,label='reliable')
plt.scatter(urelSpks1,np.ones(len(urelSpks1)),color='cyan',alpha=0.5,label='unreliable')
plt.scatter(tmpRel,np.zeros(len(tmpRel)),color='black',facecolors='none')
plt.scatter(tmpUrel,np.ones(len(tmpUrel)),color='black',facecolors='none')
plt.xlabel('time',fontsize=25)
plt.yticks([])
plt.title('cell 1')
plt.ylabel('spikes',fontsize=20)
plt.xticks(fontsize=15)
plt.legend(loc='center right')
plt.savefig('relVunrel_partitioned.png',bbox_inches='tight',dpi=200)
plt.show()
```

6.5. Python code: get the rates and stimulus for the scatter plots shown in figures 9 and 10.

```
import matplotlib.pyplot as plt
import numpy as np

###### GET THE AVERAGE FIRING RATE OVER EACH SPIKE TRAIN #######

j=0
urelDict={}
for t in tmp:
    urelDict[str(j)] = urelSpks1[urelSpks1<t]
    urelSpks1 = urelSpks1[urelSpks1>t]
    j+=1
print('number of rates = '+str(j))
for jj in range(j):
    plt.scatter(urelDict[str(jj)],np.zeros(len(urelDict[str(jj)])))
    tr = urelDict[str(jj)]
    if len(tr)>1:
        rate = len(tr)/(np.max(tr)-np.min(tr))
    rateVec = np.append(rateVec,rate)
    #print('rate = '+str(rate))
rateVec = list(rateVec)
# remove consecutive repeats
l = len(rateVec)
k=0
while k < l-1:
    if rateVec[k] == rateVec[k+1]:
        rateVec = list(rateVec)
        rateVec.pop(k)
    l = len(rateVec)
    k+=1
print(rateVec)
rateVec = np.array(rateVec)


###### GET THE AVERAGE VALUE OF THE STIMULUS OVER THE DISTINCT SPIKE TRAINS #######
tvec = np.arange(0,tmax,dt)
#plt.plot(tvec,I)
ints = {}
i = 0
for jj in range(j):
    if len(urelDict[str(jj)]) > 0:
        ints[str(i)] = np.array([int(urelDict[str(jj)][0]/dt), int(urelDict[str(jj)][-1])/dt])
        i+=1

for ii in range(i):
    #print(ints[str(ii)])
    interval = np.array(ints[str(ii)],dtype=int)
```

```
    #print(interval[0])
    plt.plot(tvec[interval[0]:interval[1]],I[interval[0]:interval[1]])
    if interval[0] != interval[1]:
        mean = np.mean(I[interval[0]:interval[1]])
#     else:
#         mean = I[interval[0]]
    stimVec = np.append(stimVec,mean)
    print('mean = '+str(mean))
```

6.6. Python code: plot the rates and stimuli with no time lag, compute the mutual information to produce figure 9.

```
import numpy as np
from sklearn.feature_selection import mutual_info_regression
mi = mutual_info_regression(np.array([rateVec]).T, stimVec)
print(mi)
plt.title('MI = {:.2f}'.format(mi[0]),fontsize=30)
plt.scatter(rateVec,stimVec, edgecolor="black", s=50)
plt.xticks([])
plt.yticks([])
plt.xlabel('reliable spike rates',fontsize=25)
plt.ylabel('avg. stim. value',fontsize=25)
plt.savefig('MI_rel_.png',bbox_inches='tight',dpi=200)
plt.show()
```

6.7. Python code: plot the rates and stimuli a 9-spike-train time lag, compute the mutual information to produce figure 10.

```
import numpy as np
from sklearn.feature_selection import mutual_info_regression
lag = 9
mi = mutual_info_regression(np.array([rateVec[lag:]]).T, stimVec[:-lag])
print(mi)
plt.title('MI = {:.3f}'.format(mi[0]),fontsize=30)
plt.scatter(rateVec[lag:],stimVec[:-lag], edgecolor="black", s=50)
plt.xticks([])
plt.yticks([])
plt.xlabel('reliable spike rates',fontsize=25)
plt.ylabel('previous \navg. stim. value',fontsize=25)
plt.savefig('MI_rel_lag.png',bbox_inches='tight',dpi=200)
plt.show()
```

6.8. Python code: functions for large network simulation.

```python
import numpy as np
import matplotlib.pyplot as plt
from numba import jit

def make_A(num=1000):
    '''
    num = number of neurons
    '''
    K = int(0.02*num) # average number of incoming connections
    A = np.zeros((num,num),dtype=float) # weight matrix
    EorIvec = np.array([],dtype=bool)
    for iPre in range(num): # loop through pre-synaptic neurons
        EorI = np.random.uniform(0,1) # coin flip to determine whether this neuron is excitatory or inhibitory
        if EorI < 0.8: # make this neuron excitatory
            EorIvec = np.append(EorIvec,True) # True means excitatory
        else:
            EorIvec = np.append(EorIvec,False) # False means inhibitory
    EorIvec = 2*EorIvec-1
```

```python

    for iPost in range(num): # loop through post-synaptic cells
        numIn = K
        #numIn = random.randint(K-5,K+5) # number of incoming connections
        #print('iPost = '+str(iPost))
        #print('numIn = '+str(numIn))
        numInCounter = 0
        while numInCounter < numIn: # assign connections until there are numIn of them
            iPre = np.random.randint(0,num) # random pre-synaptic index
            if iPost != iPre: # no self-connections allowed, excitatory pre
                weight = EorIvec[iPre]/np.sqrt(K) # set the weight with the 1/sqrt(K) scaling
                A[iPre][iPost] = weight
                numInCounter += 1
    return A, EorIvec

# full network, return phases
@jit
def numba_LJ_net(tVec,I,eta,eps,A):
    def g(theta):
        b=1/20
        d=35/32
        if isinstance(theta, float):
            condition = (theta>-b) and (theta<b)
            if condition:
                return d*(b**2 - ((theta+.5) % 1 - .5)**2)**3
            else:
                return 0.
        else:
            ret = np.zeros(len(theta))
            for i in range(len(theta)):
                condition = (theta[i]>-b) and (theta[i]<b)
                if condition:
                    ret[i] = d*(b**2 - ((theta[i]+.5) % 1 - .5)**2)**3
                else:
                    ret[i] = 0.
            return ret
    '''
    Use Euler-Maruyama method to solve LJ eqn for full network.
    A = weight matrix
    '''
    numNrns = np.shape(A)[0]
    epsilon = eps
    dt = tVec[-1]-tVec[-2]
    phi0 = np.random.rand(numNrns)
    N = len(tVec)
    phi = np.zeros((N,numNrns))
    phi[0]=phi0
    for n in range(N-1): # loop through time
        for j in range(numNrns): # loop through neurons
            if phi[n][j] > 1: # spike
                phi[n+1][j] = 0.
            else:
                F = (1 + np.cos(2*np.pi*phi[n][j]))/(np.pi)
                Z = (1 - np.cos(2*np.pi*phi[n][j]))/(np.pi)
                #Zp=0
                Zp= np.pi*np.sin(2*np.pi*phi[n][j])
#                 F = (1 + np.cos(2*np.pi*phi[n][j]))/(np.pi)
#                 Z = (1 - np.cos(2*np.pi*phi[n][j]))/(np.pi)
#                 Zp= np.pi*np.sin(2*np.pi*phi[n][j])
                #dphidt = F + (eta + np.sum(A[:,j]*0.000001))*Z + Z*Zp*epsilon**2/2
                dphidt = F + (eta + np.sum(A[:,j]*g(phi[n])))*Z + Z*Zp*epsilon**2/2
                phi[n+1][j] = phi[n][j] + dphidt*dt + epsilon*I[n][j]*Z
    return phi

```

```python
@jit
def numba_LJ_net_st(tVec,I,eta,eps,A):
    def g(theta):
        b=1/20
        d=35/32
        if isinstance(theta, float):
            condition = (theta>-b) and (theta<b)
            if condition:
                return d*(b**2 - ((theta+.5) % 1 - .5)**2)**3
            else:
                return 0.
        else:
            ret = np.zeros(len(theta))
            for i in range(len(theta)):
                condition = (theta[i]>-b) and (theta[i]<b)
                if condition:
                    ret[i] = d*(b**2 - ((theta[i]+.5) % 1 - .5)**2)**3
                else:
                    ret[i] = 0.
            return ret
    '''
    Use Euler-Maruyama method to solve LJ eqn for full network.
    A = weight matrix
    '''
    numNrns = np.shape(A)[0]
    epsilon = eps
    phi0 = np.random.rand(numNrns)
    N = len(tVec)
    dt = tVec[-1]-tVec[-2]
    phi = np.zeros((N,numNrns))
    phi[0]=phi0
    # for raster plot
    spikeTimes = []
    spikeNeurons = []
    #spikeTimes=np.array([],dtype=float) # to store the time of each spike
    #spikeNeurons=np.array([],dtype=float) # to store the neuron index of each spike
    for n in range(N-1): # loop through time
        for j in range(numNrns): # loop through neurons
            if phi[n][j] > 1: # spike
                phi[n+1][j] = 0.
                spikeTimes.append(n*dt)
                spikeNeurons.append(j)
                #spikeTimes = np.append(spikeTimes,n*dt) # record the spike time
                #spikeNeurons = np.append(spikeNeurons,j) # record the index of the neuron that spiked
            else:
                F = (1 + np.cos(2*np.pi*phi[n][j]))
                Z = (1 - np.cos(2*np.pi*phi[n][j]))
                Zp= 2*np.pi*np.sin(2*np.pi*phi[n][j])
                #dphidt = F + (eta + np.sum(A[:,j]*0.000001))*Z + Z*Zp*epsilon**2/2
                dphidt = F + (eta + np.sum(A[:,j]*g(phi[n])))*Z + Z*Zp*epsilon**2/2
                phi[n+1][j] = phi[n][j] + dphidt*dt + epsilon*I[n][j]*Z
    return spikeTimes, spikeNeurons
```

6.9. Python code: figure 8(A).

```python
import numpy as np
import matplotlib.pyplot as plt
from funcs import make_A
from funcs import numba_LJ_net_st

A, EorIvec = make_A()
numNrns = np.shape(A)[0]
tf=15
```

```
dt = 0.01
eta = -0.5
eps = 0.5 # 0.5 or 0.18
tVec = np.arange(0,tf,dt)
N = int(tf/dt)
np.random.seed(1244)
I = np.random.normal(loc=0.0, scale=np.sqrt(dt), size=(N,numNrns))
spikeTimes, spikeNeurons = numba_LJ_net_st(tVec,I,eta,eps,A)
tf=50
N = int(tf/dt)
np.random.seed(1244)
I = np.random.normal(loc=0.0, scale=np.sqrt(dt), size=(N,numNrns))
spikeTimes, spikeNeurons = numba_LJ_net_st(tVec,I,eta,eps,A)
spikeTimes = np.array(spikeTimes,dtype=float)
spikeNeurons = np.array(spikeNeurons,dtype=int)
excRates = []
inhRates = []
for nrn in range(numNrns):
    if EorIvec[nrn]==1: #excitatory
        excRates = np.append(excRates,len(spikeTimes[spikeNeurons==nrn])) # append the spike count
    else: # inhibitory
        inhRates = np.append(inhRates,len(spikeTimes[spikeNeurons==nrn])) # append the spike count
#plt.hist(excRates/15,density=True,label='excit',color='black',histtype='step')
#plt.hist(inhRates/15,density=True,label='inhib',color='red',histtype='step')
n, xexc, _ = plt.hist(excRates/tf, density=True)
n, xinh, _ = plt.hist(inhRates/tf, density=True)
plt.close()
import scipy.stats as stats

#noise = np.random.normal(0, 1, (1000, ))
excdensity = stats.gaussian_kde(excRates/tf)
plt.plot(xexc, excdensity(xexc),color='black',label='excit',linewidth=3)
inhdensity = stats.gaussian_kde(inhRates/tf)

plt.plot(xinh, inhdensity(xinh),color='red',label='inhib',linewidth=3)
plt.legend(fontsize=20)
plt.xticks([np.min(xexc),0.3],fontsize=25)
plt.yticks([])
plt.ylabel('Prob. density',fontsize=30)
plt.xlabel('Firing rate',fontsize=30)
plt.savefig('fig1a.png',bbox_inches='tight',dpi=200)
plt.show()
```

6.10. Python code: figure 8(B).

```
import numpy as np
import matplotlib.pyplot as plt
from funcs import make_A
from funcs import numba_LJ_net_st

A = make_A()[0]
numNrns = np.shape(A)[0]
tf=50
dt = 0.02
eta = -0.5
eps = 0.5 # 0.5 or 0.18
tVec = np.arange(0,tf,dt)
N = int(tf/dt)
np.random.seed(1244)
I = np.random.normal(loc=0.0, scale=np.sqrt(dt), size=(N,numNrns))
spikeTimes, spikeNeurons = numba_LJ_net_st(tVec,I,eta,eps,A)
spikeTimes = np.array(spikeTimes,dtype=float)
spikeNeurons = np.array(spikeNeurons,dtype=int)
```

```
CVvec = np.zeros(numNrns)
j=0
ISIvec = []
for nrn in range(numNrns):
    ISIs = np.diff(spikeTimes[spikeNeurons==nrn])
    ISIvec = np.append(ISIvec,ISIs)
    #ISIvec.append(ISIs)
    mean = np.mean(ISIs)
    sd = np.std(ISIs)
    CVvec[j] = mean/sd
    j+=1
plt.hist(ISIvec,bins=100,density=True)
plt.yticks([])
plt.ylabel('Prob. density',fontsize=25)
plt.xlabel('ISI',fontsize=25)
plt.xlim([0,15])
plt.xticks(fontsize=20)
plt.savefig('fig1b.png',bbox_inches='tight',dpi=200)
print('<CV> = '+str(np.mean(CVvec)))
```

6.11. Python code: figure 8(C).

```
import numpy as np
import matplotlib.pyplot as plt
from funcs import make_A
from funcs import numba_LJ_net

A = make_A()[0]
numNrns = np.shape(A)[0]
tf=50
dt = 0.01
eta = -0.5
eps = 0.5 # 0.5 or 0.18
tVec = np.arange(0,tf,dt)
N = int(tf/dt)
np.random.seed(1244)
I = np.random.normal(loc=0.0, scale=np.sqrt(dt), size=(N,numNrns))
phi = numba_LJ_net(tVec,I,eta,eps,A)
# # make and save figure
plt.hist(phi.flatten(),density=True,bins=200)
plt.xlim([0,1])
plt.xlabel(r'$\theta$',fontsize=30)
plt.xticks([0,0.25,0.5,0.75,1.0],fontsize=25)
plt.yticks([])
plt.xticks(fontsize=12)
plt.ylabel('Prob. density',fontsize=30)
plt.savefig('fig1c.png',bbox_inches='tight',dpi=200)
plt.show()
```

6.12. Python code: figure 8(C) inset.

```
import numpy as np
import matplotlib.pyplot as plt
from funcs import make_A
from funcs import numba_LJ_net

A = make_A()[0]
numNrns = np.shape(A)[0]
tf=50
dt = 0.01
eta = -0.5
eps = 0.5 # 0.5 or 0.18
```

```
tVec = np.arange(0,tf,dt)
N = int(tf/dt)
np.random.seed(1244)
I = np.random.normal(loc=0.0, scale=np.sqrt(dt), size=(N,numNrns))
phi = numba_LJ_net(tVec,I,eta,eps,A)
# # make and save the inset figure
plt.xlim([0,tf])
nrnInd = np.random.randint(0,numNrns) # pick a random neuron index
plt.plot(tVec, phi[:,nrnInd],color='black',linewidth=2)
plt.hlines(.3,0,50,linestyle='-',color='red',linewidth=3) # stable FP
plt.hlines(.7,0,50,linestyle='--',color='red',linewidth=3) # unstable FP
plt.ylim([0,1])
plt.xticks([0,50],fontsize=25)
plt.yticks([0,1],fontsize=25)
plt.xlabel('time',fontsize=30)
plt.ylabel(r'$\theta$',fontsize=30)
plt.savefig('fig1c_inset.png',bbox_inches='tight',dpi=200)
plt.show()
```

6.13. Python code: figure 8(D).

```
import numpy as np
import matplotlib.pyplot as plt
from funcs import make_A
from funcs import numba_LJ_net_st

A, EorIvec = make_A()
numNrns = np.shape(A)[0]
tf=15
dt = 0.01
eta = -0.5
eps = 0.5 # 0.5 or 0.18
tVec = np.arange(0,tf,dt)
N = int(tf/dt)
np.random.seed(1244)
I = np.random.normal(loc=0.0, scale=np.sqrt(dt), size=(N,numNrns))
spikeTimes, spikeNeurons = numba_LJ_net_st(tVec,I,eta,eps,A)
spikeTimes = np.array(spikeTimes,dtype=float)
spikeNeurons = np.array(spikeNeurons,dtype=int)
numNrns = np.shape(A)[0]
nrnInd = np.random.randint(0,numNrns) # pick a random neuron index
plt.scatter(spikeTimes[spikeNeurons<250],spikeNeurons[spikeNeurons<250],s=1.5,color='black')
plt.xlim([0,15])
plt.ylim([0,250])
plt.xlabel(r'time, $t$',fontsize=30)
plt.ylabel(r'neuron index',fontsize=30)
plt.xticks([0,15],fontsize=25)
plt.yticks(fontsize=25)
plt.savefig('fig1d.png',bbox_inches='tight',dpi=200)
plt.show()
```

References

[1] A. A. Faisal, L. P. Selen, D. M. Wolpert, Noise in the nervous system, Nature reviews neuroscience 9 (4) (2008) 292–303.

[2] K. K. Lin, E. Shea-Brown, L.-S. Young, Reliability of coupled oscillators, Journal of nonlinear science 19 (5) (2009) 497–545.

[3] G. Lajoie, K. K. Lin, E. Shea-Brown, Chaos and reliability in balanced spiking networks with temporal drive, Physical Review E 87 (5) (2013) 052901.

[4] P. Dayan, L. F. Abbott, Theoretical neuroscience: computational and mathematical modeling of neural systems, MIT press, 2005.

[5] C. Koch, Biophysics of computation: information processing in single neurons, Oxford university press, 2004.
[6] R. Pyle, R. Rosenbaum, Spatiotemporal dynamics and reliable computations in recurrent spiking neural networks, Physical review letters 118 (1) (2017) 018103.
[7] J. Schuecker, S. Goedeke, M. Helias, Optimal sequence memory in driven random networks, Physical Review X 8 (4) (2018) 041029.
[8] M. Nolte, M. W. Reimann, J. G. King, H. Markram, E. B. Muller, Cortical reliability amid noise and chaos, Nature communications 10 (1) (2019) 1–15.

School of Mathematics, Georgia Institute of Technology, Atlanta, GA USA 30332